

Experiment 1 | Lab Report

Basic Operations Using GPIO: Blinking LEDs

Interfacing four LEDs with a Raspberry Pi and an ESP32

Submitted by	Chavan Atharva Sunil
Roll Number	240003021
Date of Experiment	6 August 2026
Boards Used	Raspberry Pi (BCM numbering) • ESP32 DevKit

1 Aim

To understand the basic operation of GPIO (General Purpose Input/Output) pins by interfacing LEDs with a **Raspberry Pi** and an **ESP32**. Four LEDs are driven in two patterns — all four blinking **simultaneously**, and a **sequential** pattern with one LED ON at a time — and the setup is then extended to **eight LEDs** that display a decimal number, entered by the user at the terminal or serial monitor, as its **8-bit binary** equivalent.

Objectives

- To identify the GPIO header of the Raspberry Pi and the ESP32 and select pins that are safe to use as general-purpose digital outputs.
- To configure the selected GPIO pins in *output* mode from software and to wire four LEDs on a breadboard with a common ground return.
- To write, execute and verify blink programs in **Python** (RPi.GPIO) and in **C/C++** (Arduino core for ESP32).
- To display a user-entered decimal number (0–255) on eight LEDs by converting it to binary and mapping each bit to one GPIO pin.
- To observe and record the resulting output.

Brief Theory

A GPIO pin is a pin on a microcontroller or single-board computer whose direction and logic level are controlled entirely by software. When a pin is configured as a digital output and written HIGH, the pin is internally connected to the supply rail (3.3 V on both the Raspberry Pi and the ESP32); when written LOW it is connected to ground. Connecting an LED between such a pin and GND therefore lets the program switch the LED on and off. Toggling the pin state in a loop, with a delay between transitions, produces a visible blink; toggling four pins inside the same iteration makes all four LEDs change state together.

Note on current limiting. In this experiment the LEDs were connected *directly* to the GPIO pins, without series resistors, and the brief demonstration ran without damage. Each GPIO pin can nevertheless source only a limited current (about 16 mA on the Raspberry Pi and 12 mA on the ESP32), and with a red LED whose forward drop is ≈ 2 V the current is bounded only by the LED and the pin's own internal resistance. For sustained use a series resistor of roughly $220\,\Omega$ per LED — giving $(3.3 - 2)/220 \approx 6$ mA — is the recommended practice.

2 Materials Required

#	Component	Qty.	Purpose in the setup
1	Raspberry Pi (with SD card, OS installed)	1	Host board for Experiment 1
2	ESP32 development board	1	Host board for Experiment 2
3	Solderless breadboard	1	Mounting the LEDs and the ground rail
4	LEDs (5 mm, red)	8	Output indicators; 4 for the blink patterns, 8 for the binary display
5	Male–female jumper wires	16–20	GPIO header to breadboard connections
6	USB power supply / USB cable	1	Powering and programming the board
7	Computer (with Arduino IDE)	1	Writing, compiling and uploading the code

The Raspberry Pi was operated with a monitor, keyboard and mouse; the ESP32 was programmed over USB from the Arduino IDE with the board set to ESP32 Dev Module.

3 Methodology

The experiment was carried out in two parts on two different platforms. In both parts the electrical arrangement is identical — only the pin numbers and the programming environment differ.

3.1 Experiment 1 — Blinking Four LEDs Using Raspberry Pi GPIO

Four LEDs were placed on the breadboard, each connected by a jumper wire directly to a separate GPIO pin of the Raspberry Pi. The cathodes of all four LEDs were tied to a common ground rail on the breadboard, which was returned to a GND pin of the Pi.

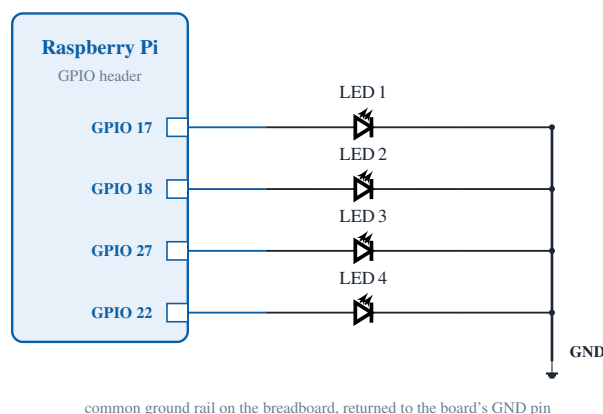
All four GPIO pins were then configured as digital outputs in software. Inside an infinite loop, the four pins were set HIGH at the same time — turning all four LEDs ON — held for a fixed delay of one second, then set LOW at the same time, turning all four LEDs OFF for another second. Repeating this cycle continuously produces the blinking effect.

GPIO connections

LED	Raspberry Pi GPIO	Physical pin
LED 1	GPIO 17	Pin 11
LED 2	GPIO 18	Pin 12
LED 3	GPIO 27	Pin 13
LED 4	GPIO 22	Pin 15
All cathodes	GND	Pin 6 / Pin 9

Pin numbering

The program uses **BCM** numbering (`GPIO.setmode(GPIO.BCM)`), i.e. the Broadcom channel numbers printed in the pinout diagram, *not* the physical header positions. The physical pin column is given only for wiring convenience.



common ground rail on the breadboard, returned to the board's GND pin

Figure 1: Circuit schematic for Experiment 1. Each GPIO pin drives one LED directly; all four cathodes return to the common GND rail on the breadboard.

Procedure

1. The four LEDs were placed on the breadboard, with the anode (longer leg) and cathode (shorter leg) of each LED in separate rows.
2. The anode of each LED was taken by a jumper wire to a separate GPIO pin of the Raspberry Pi (GPIO 17, 18, 27 and 22); no series resistors were used. The wiring was checked with the board powered off.
3. The cathodes of all four LEDs were connected to the common ground rail, which was wired back to a GND pin on the Pi.
4. The four GPIO pins were configured as digital output pins in the Python program using `GPIO.setup()`.
5. The program was executed on the Raspberry Pi.
6. All four GPIO pins were set HIGH simultaneously, turning ON all four LEDs.
7. After a fixed delay of one second, all four GPIO pins were set LOW simultaneously, turning OFF all four LEDs.
8. The process was repeated continuously until the program was interrupted, after which `GPIO.cleanup()` released the pins.

Code (Python, RPi.GPIO)

Listing 1: Blinking four LEDs simultaneously on the Raspberry Pi.

```

1 import RPi.GPIO as GPIO
2 import time
3
4 LED_PINS = [17, 18, 27, 22] # BCM channel numbers
5
6 GPIO.setmode(GPIO.BCM)      # use Broadcom pin numbering
7 for pin in LED_PINS:        # configure every pin as a digital output
8     GPIO.setup(pin, GPIO.OUT)
9
10 try:
11     while True:
12         # Turn all LEDs ON
13         for pin in LED_PINS:
14             GPIO.output(pin, GPIO.HIGH)
15         time.sleep(1)
16
17         # Turn all LEDs OFF
18         for pin in LED_PINS:
19             GPIO.output(pin, GPIO.LOW)
20         time.sleep(1)
21
22 except KeyboardInterrupt:
23     pass
24 finally:
25     GPIO.cleanup()           # reset all pins to a safe state on exit

```

Working

All four LEDs are controlled together. When the GPIO pins are set HIGH, all four LEDs turn ON; when the GPIO pins are set LOW, all four LEDs turn OFF. Because the four `GPIO.output()` calls execute within microseconds of one another, the transitions appear perfectly simultaneous to the eye.

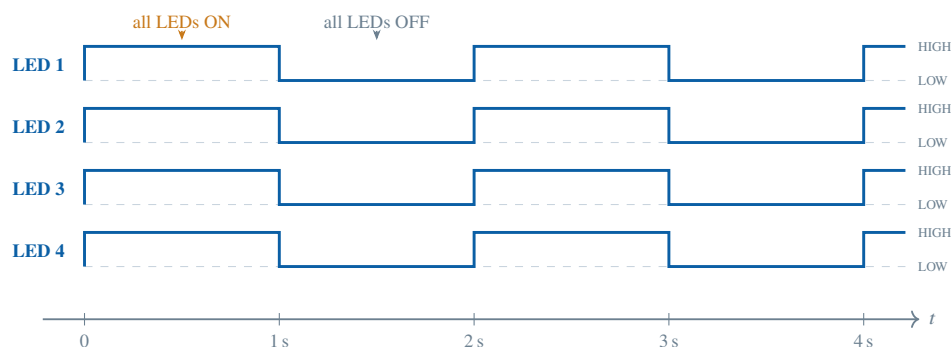


Figure 2: Timing diagram of the four GPIO outputs. The four waveforms are identical and exactly in phase, which is what makes the LEDs blink simultaneously.

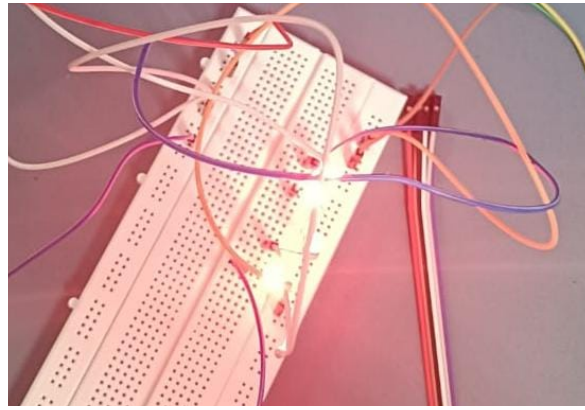


Figure 3: Experiment 1 — all four LEDs lit simultaneously during the ON phase of the blink cycle, with the jumper wires running to the GPIO header and all cathodes on the common ground rail.

3.2 Experiment 2 — Blinking Four LEDs Using ESP32

The same four LEDs were re-wired to four GPIO pins of the ESP32 development board, again connected directly, with all cathodes on the common ground rail returned to the ESP32's GND pin.

The four GPIO pins were configured as digital outputs inside `setup()`. Within `loop()`, all four pins were driven HIGH simultaneously to turn the LEDs ON, held for one second using `delay(1000)`, then driven LOW simultaneously to turn them OFF for another second. Since `loop()` is called repeatedly by the Arduino core, the blinking continues indefinitely.

Additional materials

- ESP32 development board, USB cable, and a computer with the Arduino IDE (ESP32 board support package installed).

GPIO connections

LED	ESP32 GPIO	Board label
LED 1	GPIO 2	D2
LED 2	GPIO 4	D4
LED 3	GPIO 5	D5
LED 4	GPIO 18	D18
All cathodes	GND	GND

Choice of pins

GPIO 2, 4, 5 and 18 are ordinary output-capable pins on the ESP32 and are free of boot-strapping restrictions in this configuration. Input-only pins (GPIO 34–39) cannot drive an LED and were therefore avoided.

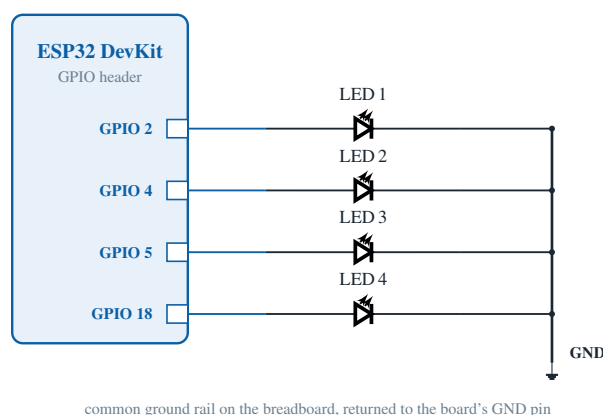


Figure 4: Circuit schematic for Experiment 2. The topology is identical to Figure 1; only the driving pins change.

Procedure

1. The four LEDs were placed on the breadboard, anodes in separate rows.
2. Each LED branch was connected to a separate GPIO pin of the ESP32 (GPIO 2, 4, 5 and 18) and all cathodes were returned to the GND pin through the common ground rail.

3. The ESP32 was connected to the computer over USB and the correct board and COM port were selected in the Arduino IDE.
4. The program was written and uploaded, configuring all four GPIO pins as outputs in `setup()`.
5. All four LEDs were switched ON simultaneously.
6. After a one-second delay, all four LEDs were switched OFF simultaneously.
7. The process repeated continuously as `loop()` re-executed.

Code (Arduino C++ for ESP32)

Listing 2: Blinking four LEDs simultaneously on the ESP32.

```

1  int LED1 = 2;
2  int LED2 = 4;
3  int LED3 = 5;
4  int LED4 = 18;
5
6  void setup() {
7      pinMode(LED1, OUTPUT);
8      pinMode(LED2, OUTPUT);
9      pinMode(LED3, OUTPUT);
10     pinMode(LED4, OUTPUT);
11 }
12
13 void loop() {
14     // Turn all LEDs ON
15     digitalWrite(LED1, HIGH);
16     digitalWrite(LED2, HIGH);
17     digitalWrite(LED3, HIGH);
18     digitalWrite(LED4, HIGH);
19     delay(1000);
20
21     // Turn all LEDs OFF
22     digitalWrite(LED1, LOW);
23     digitalWrite(LED2, LOW);
24     digitalWrite(LED3, LOW);
25     digitalWrite(LED4, LOW);
26     delay(1000);
27 }

```

Working

The four LEDs operate simultaneously. When all four GPIO pins are HIGH, all four LEDs turn ON; when all four GPIO pins are LOW, all four LEDs turn OFF. The behaviour is identical to Figure 2, confirming that the same GPIO concept transfers unchanged from a Linux single-board computer to a microcontroller.

3.3 Pattern 2 — One LED ON at a Time

After verifying the simultaneous blink, the same wiring was used for a **sequential** pattern in which exactly one LED is lit at any instant. Only the program changes: instead of writing the same level to all four pins, the loop walks an index through the pin list, driving one pin HIGH while the other three are held LOW, then advancing after a one-second delay. This produces a running-light (“chaser”) effect across LED 1 → LED 2 → LED 3 → LED 4 and back to the start.

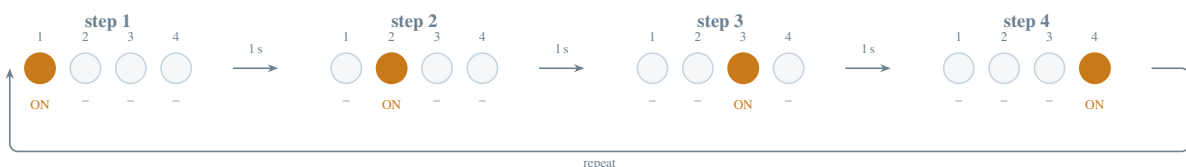


Figure 5: The sequential pattern: exactly one LED is ON in each step, and the lit position advances once per second before wrapping back to LED 1.

Listing 3: Sequential pattern on the Raspberry Pi — one LED ON at a time.

```

1 while True:
2     for on_pin in LED_PINS:           # walk the lit position along the list
3         for pin in LED_PINS:
4             GPIO.output(pin, GPIO.HIGH if pin == on_pin else GPIO.LOW)
5             time.sleep(1)

```

On the ESP32 the same logic was used inside `loop()`: the four `digitalWrite()` calls set one pin HIGH and the remaining three LOW, followed by `delay(1000)` before the lit position advances.

3.4 Experiment 3 — Eight-LED Binary Number Display

In the final part of the experiment the setup was extended from four LEDs to **eight**, and the LEDs were used not as a blinking pattern but as an **8-bit output register**. The user types a decimal number in the range 0–255 — on the Raspberry Pi at the terminal, on the ESP32 in the Arduino IDE Serial Monitor — and the program converts that number to its 8-bit binary form and drives one LED per bit: a LED is ON where the bit is 1 and OFF where it is 0. The row of eight LEDs therefore reads directly as the binary value of the number entered.

GPIO connections

Bit (weight)	Raspberry Pi	ESP32
bit 7 (128)	Pin 40	GPIO 23
bit 6 (64)	Pin 38	GPIO 22
bit 5 (32)	Pin 36	GPIO 21
bit 4 (16)	Pin 35	GPIO 19
bit 3 (8)	Pin 33	GPIO 18
bit 2 (4)	Pin 32	GPIO 5
bit 1 (2)	Pin 31	GPIO 4
bit 0 (1)	Pin 29	GPIO 2
All cathodes	GND	GND

Numbering used here

The Raspberry Pi program for this part uses **BOARD** numbering (`GPIO.setmode(GPIO.BOARD)`), so the numbers in the table are the *physical* header positions rather than the BCM channels used in Experiment 1. The ESP32 column gives GPIO numbers as before. The most significant bit is wired at the top of the row so that the LEDs read left-to-right in the same order as the printed binary string.

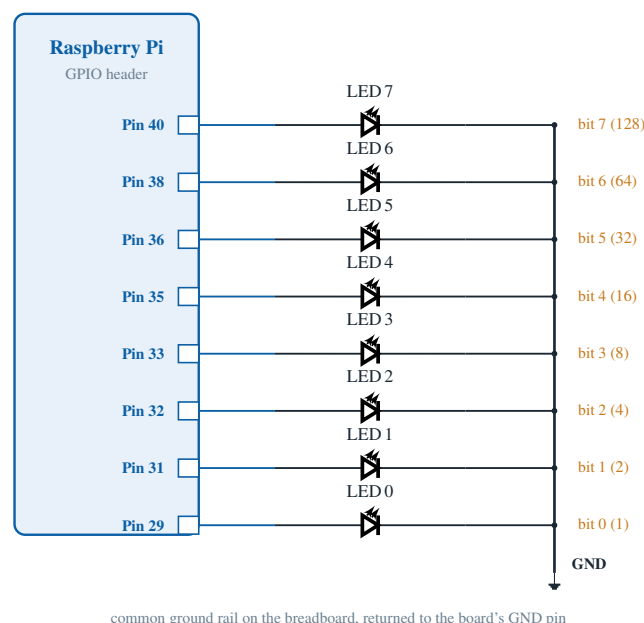


Figure 6: Circuit schematic for Experiment 3. Eight LEDs are driven directly from eight GPIO pins, one per bit, with all cathodes on the common ground rail. The ESP32 wiring is identical, using GPIO 23, 22, 21, 19, 18, 5, 4 and 2 in place of the header pins.

Principle of operation

Each LED is assigned a place value: the leftmost carries $2^7 = 128$ and the rightmost $2^0 = 1$. A number is displayed by testing its bits one at a time and writing the result to the matching pin, so the pattern of lit LEDs is the binary representation of the number. For example, $200 = 128 + 64 + 8$, giving the bit pattern 11001000:

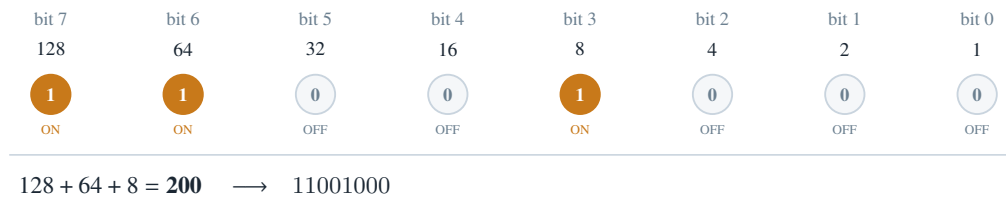


Figure 7: Place values of the eight LEDs and the pattern displayed for the input 200. The same mapping was confirmed on the serial monitor for the inputs $20 \rightarrow 00010100$ and $25 \rightarrow 00011001$.

Procedure

1. Eight LEDs were placed in a row on the breadboard with all cathodes on the common ground rail, which was returned to the board's GND pin.
2. Each anode was wired to its own GPIO pin in the order given in the table above, the most significant bit first.
3. The program was run; it prompts for a decimal number on the terminal (Raspberry Pi) or on the Serial Monitor (ESP32).
4. The entered number was converted to an 8-bit binary string, zero-padded on the left so that it always has exactly eight digits.
5. Each bit was written to its corresponding GPIO pin, lighting the LEDs that correspond to the set bits.
6. The binary string was also printed back to the console so that the display could be checked against the computed value.
7. Steps 3–6 were repeated for several inputs, including 200, 20 and 25.

Code (Python, Raspberry Pi)

Listing 4: Displaying a decimal number on eight LEDs as 8-bit binary (Raspberry Pi).

```

1 import RPi.GPIO as GPIO
2
3 # Physical (BOARD) pin numbers, most significant bit first
4 LED_PINS = [40, 38, 36, 35, 33, 32, 31, 29]
5
6 GPIO.setmode(GPIO.BOARD)
7 GPIO.setwarnings(False)
8 for pin in LED_PINS:
9     GPIO.setup(pin, GPIO.OUT, initial=GPIO.LOW)
10
11 try:
12     while True:
13         number = int(input("Enter a number (0-255): "))
14         if not 0 <= number <= 255:
15             print("Out of range - please enter a value from 0 to 255.")
16             continue
17
18         bits = format(number, '08b') # zero-padded to exactly 8 digits
19         print(number, "->", bits)
20
21         for pin, bit in zip(LED_PINS, bits):
22             GPIO.output(pin, GPIO.HIGH if bit == '1' else GPIO.LOW)
23
24 except KeyboardInterrupt:
25     print("\nExiting program")
26
27 finally:
28     GPIO.cleanup()

```


Code (Arduino C++ for ESP32)

Listing 5: Displaying a decimal number on eight LEDs as 8-bit binary (ESP32).

```

1  const int LED_PINS[8] = {23, 22, 21, 19, 18, 5, 4, 2}; // MSB first
2
3  void setup() {
4      Serial.begin(115200);
5      for (int i = 0; i < 8; i++) {
6          pinMode(LED_PINS[i], OUTPUT);
7          digitalWrite(LED_PINS[i], LOW);
8      }
9      Serial.println("--- 8-Bit Binary LED Controller ---");
10     Serial.println("Enter a number between 0 and 255.");
11 }
12
13 void loop() {
14     if (Serial.available() > 0) {
15         String line = Serial.readStringUntil('\n');
16         line.trim();
17         if (line.length() == 0) return;
18
19         int number = line.toInt();
20         if (number < 0 || number > 255) {
21             Serial.println("Out of range - please enter a value from 0 to 255.");
22             return;
23         }
24
25         Serial.print("Number: "); Serial.print(number);
26         Serial.print(" -> Binary: ");
27
28         for (int b = 7; b >= 0; b--) {
29             int value = (number >> b) & 1; // extract one bit
30             digitalWrite(LED_PINS[7 - b], value); // drive the matching LED
31             Serial.print(value);
32         }
33         Serial.println();
34     }
35 }

```

Working

Both programs perform the same three steps in a loop: read a decimal number, convert it to a fixed-width 8-bit string, and write the eight bits to the eight GPIO pins. The Python version uses `format(number, '08b')` and the ESP32 version shifts and masks the number one bit at a time with `(number >> b) & 1`; the two are equivalent. Because the LEDs are wired most-significant-bit first, the illuminated row matches the binary string printed on the console digit for digit.

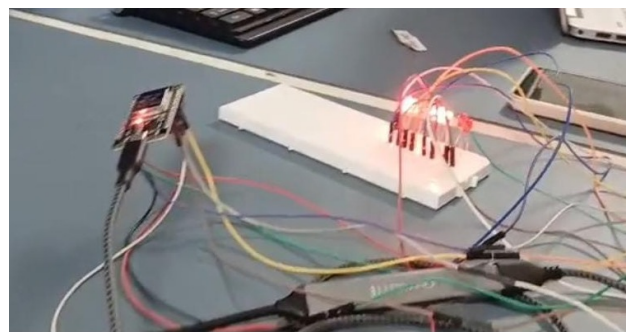
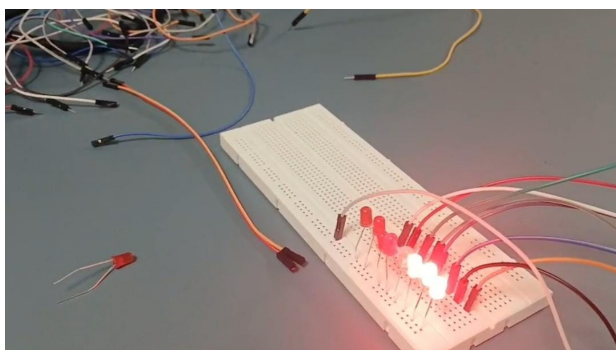


Figure 8: The eight-LED setup. *Left:* the row of eight LEDs on the breadboard displaying a bit pattern. *Right:* the complete bench arrangement, with the ESP32 on the left and the eight LEDs wired to its GPIO pins.

4 Result and Output

All three parts worked as intended on **both** the Raspberry Pi and the ESP32. The four LEDs blinked **simultaneously**, turning ON and OFF together at one-second intervals; in the second pattern exactly **one LED was ON at a time**, the lit position advancing once per second; and the eight-LED array correctly displayed every decimal number entered as its **8-bit binary** equivalent.

Observed output

Pattern	GPIO state	LED state	Duration / remark
Simultaneous	All pins HIGH	LED 1–4 ON	held 1 s; all four glow together
	All pins LOW	LED 1–4 OFF	held 1 s; all four go dark together
Sequential	One pin HIGH other three LOW	one LED ON three LEDs OFF	held 1 s, then the lit position advances 1 → 2 → 3 → 4 both cycles repeat continuously
Binary (8 LEDs)	input 200	11001000	LEDs for bits 7, 6 and 3 lit
	input 20	00010100	LEDs for bits 4 and 2 lit
	input 25	00011001	LEDs for bits 4, 3 and 0 lit

The measured blink period in the first pattern was 2 s (1 s ON + 1 s OFF), i.e. 0.5 Hz with a 50 % duty cycle; the sequential pattern completed one full sweep of the four LEDs every 4 s. Both matched the delay values written in the programs. In the binary display the lit pattern agreed with the binary string printed on the console for every value tested, and the display updated immediately on each new entry.

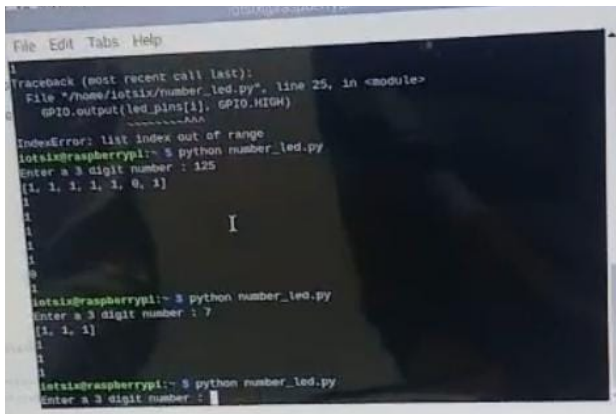
Photographic evidence

Figure 9: The terminal session on the Raspberry Pi. The lower prompts read a decimal input and print the resulting bit list; the traceback at the top is the `IndexError` raised by the earlier, unpadded version of the program.

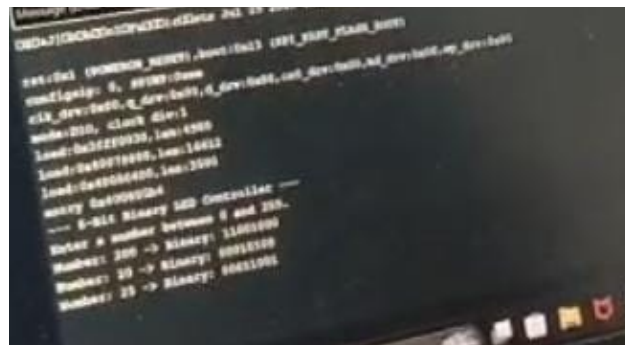


Figure 10: The Arduino IDE Serial Monitor for the ESP32, showing 200 → 11001000, 20 → 00010100 and 25 → 00011001.

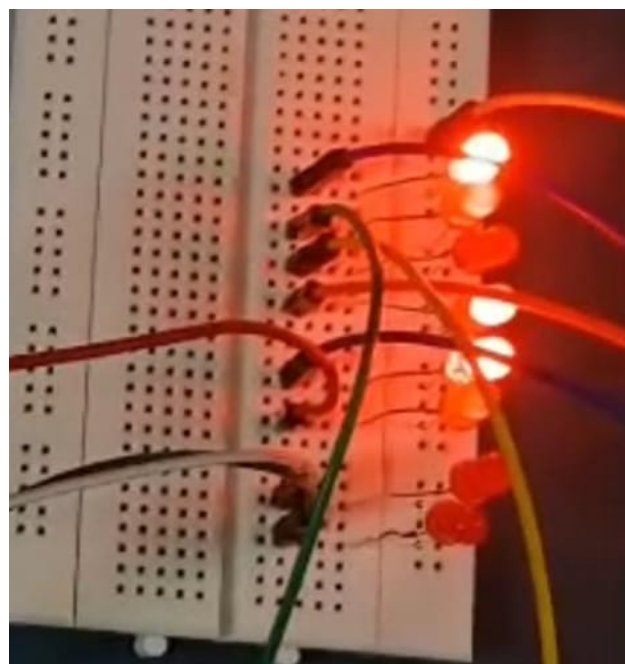
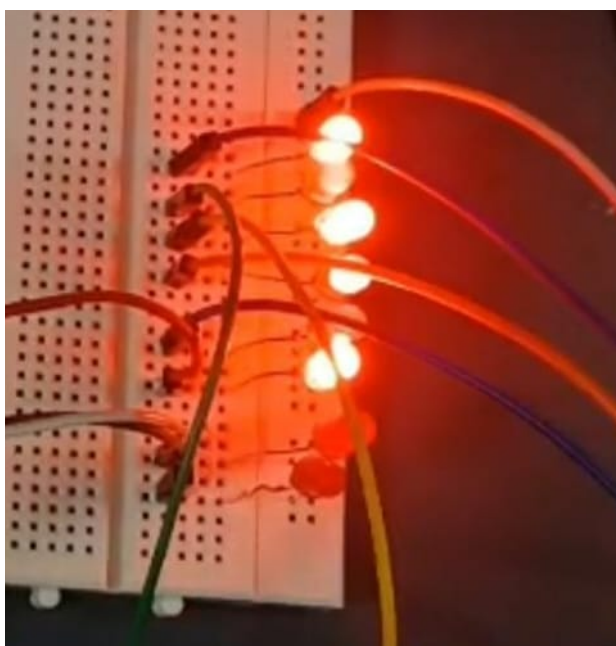


Figure 11: Two different numbers displayed on the eight-LED array; the lit LEDs correspond to the set bits of the value entered on the console.

Video demonstration and code

Google Drive link

The demonstration videos of all three parts on both boards, together with this report and the complete source code for every listing above, are available in the following shared folder:

https://drive.google.com/drive/folders/11TP-NX_EF_IpxHIAUYmG8YWPQfst2ae2?usp=sharing

Access has been set to *anyone with the link can view*.

Discussion

- The LEDs switched in unison because all four output writes occur inside the same loop iteration; the few microseconds separating them are far below the threshold of visual perception.
- Reversing an LED (cathode to the GPIO pin) leaves it permanently dark, since the diode then blocks conduction — a useful check when a single LED fails to light. Because no series resistors were used, the LEDs were driven close to the pin's current limit — acceptable for a short demonstration, though a $220\ \Omega$ resistor per LED would be the safer arrangement.
- On the Raspberry Pi, `GPIO.cleanup()` in the finally block leaves no pin driven HIGH once the program is stopped.
- In the binary display the bit string *must* be zero-padded to eight digits. An earlier version of the Raspberry Pi program built the bit list by repeated division, which yields only as many digits as the number needs, and indexing eight pins against that shorter list raised an `IndexError: list index out of range`. Using `format(number, '08b')` fixes this by always producing exactly eight characters.
- Wiring the most significant bit first means the row of LEDs can be read left-to-right in the same order as the printed binary string, which made checking the output against the console immediate.

5 Conclusion

The experiment demonstrated the basic operation of GPIO pins as digital outputs: four LEDs were interfaced with both the Raspberry Pi and the ESP32 and driven in two patterns — all four blinking together, and one LED lit at a time — at one-second intervals. The setup was then extended to eight LEDs used as an 8-bit output register, displaying any decimal number from 0 to 255 entered at the terminal or serial monitor as its binary equivalent.

The exercise established the complete path from software to hardware — selecting suitable pins, configuring them as outputs, wiring an external load to them, and controlling that load in real time — and, by producing identical behaviour on two very different platforms, confirmed GPIO control as a platform-independent foundation for the interfacing experiments that follow.